



Heimdall: Control de acceso

2º / ASIR

Adrián Cantón Alcoba

Tutor del TFG: Ignacio

DEDICATORIA

Todo este trabajo se lo quiero dedicar a mi Diego, mi padre que me inculcó la pasión por la informática desde pequeño y sin él no sé a qué me dedicaría ahora. También mención especial a mi madre por estar siempre apoyándome en todo lo que necesitaba y por esas meriendas que me daban fuerzas para dar lo mejor de mí en este trabajo.

Como no mencionar a Pepe, un compañero que a estado en todo momento donde en ningún caso hubo competición, fue mi mentor en este nuevo mundo además de su ayuda donde veía mis puntos débiles. Sin él ni siquiera sabría decir si estaría aquí ahora, gracias por todo.

ÍNDICES

Contenido

DEDICATORIA	3
Índice de ilustraciones	7
ABSTRACT	9
Inglés.....	9
JUSTIFICACIÓN DEL PROYECTO	11
Soluciones open source existentes	11
Comparativa con Heimdall:.....	12
Público objetivo	12
INTRODUCCIÓN	13
Principales funciones	13
Problemas que resuelve.....	13
OBJETIVOS.....	15
R01 – Control de Acceso Automatizado.....	15
R01F01 – Sistema Embebido y Lógica de Hardware (Arduino/ESP32)	
.....	15
R02 – Infraestructura de Red Segura.....	17
R02F01 – Segmentación de Red y Bastionado (VLAN/DMZ).....	17
R03 – Registro de Eventos y Auditoría Visual.....	17
R03F01 – Backend y Almacenamiento de Datos	17
R03F02 – Frontend y Sistema de Cámaras.....	18

DESCRIPCIÓN	20
Arquitectura de la solución.	20
Casos de uso. Incluye diagrama y tabla con:	21
Caso de uso: Iniciar sesión	21
Caso de uso: Monitorear Accesos en Tiempo Real	22
Caso de uso: Consultar Historial de Accesos	24
Caso de Uso: Alta de Nuevo Usuario y Tarjeta	25
Caso de Uso: Cerrar Sesión	27
Caso de Uso: Fichar	28
DISEÑOS	30
TECNOLOGÍA	37
LENGUAJES	37
BASE DE DATOS	38
HARDWARE	39
INFRAESTRUCTURA Y DESPLIEGUE	40
Documentación	42
Infraestructura	42
Dockerfile	42
Docker compose	42
Conexiones	42
MySQL	42
PHP	43
ESP32	43

Página web.....	44
Login.....	44
Índex.....	45
Registrar	46
Historial y tarjetas	47
Páginas no visibles.....	48
Conexión.php.....	48
Logout.php.....	49
Validar.php.....	49
Ajax accesos.php.....	49
Microcontroladores.....	49
Hardware	50
Registro	52
Validación	52
METODOLOGÍA	53
Valoración inicial	53
Tiempo real	54
Presupuesto	55
TRABAJO FUTURO.....	56
Autenticación y autorización mejoradas	56
Cifrado extremo a extremo	56
Protección ante ataques.....	56
Evolución tecnológica e innovación.....	56

CONCLUSIONES	57
REFERENCIAS	58
Infraestructura y Contenerización.....	58
Microcontroladores y Firmware	58
Tecnología de Identificación (RFID)	58
Desarrollo Web y Lógica de Servidor	59
Servidor de Aplicaciones	59
Gestión de Bases de Datos.....	59

Índice de ilustraciones

Ilustración 1:Caso de uso: Iniciar sesión.....	21
Ilustración 2 Caso de uso: Inicio de Sesión	22
Ilustración 3 Caso de uso: Monitorear Accesos en Tiempo Real.....	22
Ilustración 4 Caso de uso: Monitorear Accesos en Tiempo Real.....	23
Ilustración 5 Caso de Uso: Consultar Historial de Accesos	24
Ilustración 6 Caso de Uso: Consultar Historial de Accesos	25
Ilustración 7 Caso de Uso: Alta de Nuevo Usuario y Tarjeta	25
Ilustración 8 Caso de Uso: Alta de Nuevo Usuario y Tarjeta	26
Ilustración 9 Caso de Uso: Cerrar Sesión	27
Ilustración 10 Caso de Uso: Cerrar Sesión.....	28
Ilustración 11 Diagrama E/R	30
Ilustración 12 Tablas desde MySQL	32

Ilustración 13 Interfaz gráfica del proyecto	34
Ilustración 14 Interfaz grafica desde movil	36
Ilustración 15 barra de navegación	44
Ilustración 16 login	45
Ilustración 17 index	46
Ilustración 18 registro	47
Ilustración 19 logs	48
Ilustración 20 historial de accesos	48
Ilustración 21 Circuito de Heimdall	51
Ilustración 22 tareas propuestas	53
Ilustración 23 aproximacion de horas s1 y s2	53
Ilustración 24 aproximación de horas s1 y s2	54

ABSTRACT

Heimdall es un sistema de control de acceso físico basado en tecnología RFID. El sistema integra hardware y software para gestionar y registrar el acceso a instalaciones mediante tarjetas de identificación por radiofrecuencia.

El hardware está compuesto por un microcontrolador ESP32 y un lector RFID MFRC522, que captura el identificador único (UID) de cada tarjeta y lo envía mediante HTTP POST a un servidor web. El backend está desarrollado en PHP 8.3 con PDO sobre Apache 2.4, y utiliza MySQL 8.4 como base de datos relacional para almacenar usuarios, tarjetas y registros de acceso. Toda la infraestructura se despliega mediante Docker, con tres contenedores aislados en redes diferenciadas: uno para el servidor web, otro para el procesador PHP-FPM y otro para la base de datos, garantizando que esta última no quede expuesta al exterior.

La aplicación web ofrece un panel de control en tiempo real que muestra el último acceso concedido, actualizándose automáticamente cada 1,5 segundos mediante AJAX sin recargar la página. El acceso al panel está protegido por autenticación de administrador con contraseñas almacenadas en bcrypt. El sistema distingue entre tarjetas activas, inactivas y perdidas, registrando tanto los accesos concedidos como los intentos fallidos.

Inglés

Heimdall is a physical access control system based on RFID technology. The system integrates hardware and software to manage and log access to facilities using radio-frequency identification cards.

The hardware consists of an ESP32 microcontroller and an MFRC522 RFID reader, which captures each card's unique identifier (UID) and sends it via HTTP POST to a web server. The backend is built with PHP 8.3 and PDO on Apache 2.4, using MySQL 8.4 as the relational database to store users, cards, and access records. The entire infrastructure is deployed using Docker, with three isolated containers across separate networks — one for the web server, one for PHP-FPM, and one for the database — ensuring the database is never exposed to the outside.

The web application provides a real-time dashboard displaying the latest granted access, auto-refreshing every 1.5 seconds via AJAX without page reloads. The dashboard is protected by administrator authentication with bcrypt-hashed passwords. The system distinguishes between active, inactive, and lost cards, logging both granted access and failed attempts.

JUSTIFICACIÓN DEL PROYECTO

La motivación principal que impulsa la creación de este proyecto. Estado de la cuestión, si hay aplicaciones similares, público al que va dirigido...

Se espera una comparativa razonada.

La motivación principal de Heimdall surge de una necesidad real y creciente: los sistemas de control de acceso tradicionales basados en llaves físicas presentan limitaciones evidentes en entornos que requieren trazabilidad, escalabilidad y gestión centralizada. Una llave perdida compromete la seguridad de toda una instalación, no deja registro de quién entró ni cuándo, y su reemplazo tiene un coste físico y logístico considerable.

Hay controles de acceso RFID pero suelen ser soluciones cerradas, de alto coste de adquisición y mantenimiento, y difícilmente adaptables a entornos educativos, pequeñas empresas o proyectos de investigación con presupuesto limitado. Heimdall nace para cubrir ese espacio intermedio: un sistema funcional, seguro, de código abierto y desplegable con hardware de bajo coste.

Soluciones open source existentes

Hackaday / GitHub — proyectos RFID+ESP32 existen numerosos proyectos similares en la comunidad maker, pero la mayoría se quedan en la capa de hardware: leen el UID y lo imprimen por el puerto serie, sin backend, sin base de datos ni interfaz de gestión.

Home Assistant con integración RFID permite control de acceso domótico, pero está orientado al hogar y requiere un ecosistema complejo preinstalado.

ProjectRFID y similares en GitHub ofrecen backends básicos en Python o Node.js, pero sin contenerización, sin gestión de sesiones segura ni interfaz de administración.

Comparativa con Heimdall:

Heimdall da un paso más al integrar toda la pila en un único entorno reproducible con Docker, con separación de redes, autenticación con bcrypt, registro de intentos fallidos y dashboard en tiempo real. No es solo un prototipo de hardware sino un sistema desplegable.

Público objetivo

Entornos educativos y universitarios: laboratorios, salas de servidores, aulas de acceso restringido. Necesitan trazabilidad de accesos sin el coste de una solución empresarial.

Pequeñas y medianas empresas: almacenes, oficinas, salas de reunión de acceso limitado. Buscan una solución funcional sin atarse a un proveedor ni pagar licencias recurrentes.

INTRODUCCIÓN

El proyecto Heimdall surge como una respuesta técnica a las limitaciones de los sistemas de seguridad física tradicionales, como las llaves metálicas, que carecen de trazabilidad y representan un riesgo logístico en caso de pérdida. Este sistema propone una solución de control de acceso basada en tecnología RFID (Radio Frequency Identification) que integra hardware económico y una infraestructura de software robusta, escalable y segura.

Principales funciones

El sistema centraliza la gestión de accesos mediante las siguientes capacidades clave:

- **Identificación Automatizada:** Lectura y validación de identificadores únicos (UID) mediante tarjetas Mifare y microcontroladores ESP32.
- **Gestión en Tiempo Real:** Dashboard administrativo que se actualiza cada 1,5 segundos vía AJAX para monitorizar quién accede a las instalaciones al instante.
- **Auditoría Visual:** Registro histórico que vincula cada evento de acceso con una captura de imagen y datos del usuario.
- **Administración de Credenciales:** Control total sobre el estado de las tarjetas (activas, inactivas o perdidas) y gestión de usuarios.

Problemas que resuelve

- **Falta de Trazabilidad:** Sustituye el anonimato de las llaves físicas por un registro detallado (quién, cuándo y dónde).

- Altos Costes de Propiedad: A diferencia de las soluciones propietarias cerradas, Heimdall utiliza componentes de bajo coste y software Open Source, eliminando licencias recurrentes.
- Complejidad de Despliegue: Gracias a la contenerización con Docker, el sistema es fácilmente replicable y garantiza un entorno de ejecución idéntico en cualquier servidor.
- Vulnerabilidades de Red: Resuelve la exposición de datos mediante una arquitectura de red segmentada, asegurando que la base de datos nunca sea accesible desde el exterior.

OBJETIVOS

Listado de objetivos que se plantean resolver. Requisitos.

Se debe presentar un **RFTP** inicial para acompañar a la propuesta.

R – Requisitos: Lo que debe hacer el programa expresado en lenguaje coloquial.

F – Funciones: Desglose de las características asociadas o subrequisitos de cada requisito. Expresado en lenguaje técnico.

T – Tareas asociadas a cada funcionalidad. Deben describir completamente su alcance.

P – Pruebas. Demostración o prueba planificada para cumplir cada tarea.

R01 – Control de Acceso Automatizado

Descripción: El sistema debe permitir el paso únicamente a personas autorizadas mediante una tarjeta RFID sin intervención humana, validando los permisos localmente o contra una base de datos.

R01F01 – Sistema Embebido y Lógica de Hardware (Arduino/ESP32)

Descripción técnica: Integración de microcontroladores para lectura de UID y comunicación de red.

R01F01T01 – Ensamblaje del circuito: conectar el lector RFID y el módulo ESP32 al Arduino (Tarea 1).

R01F01T01P01 – Prueba de continuidad eléctrica con multímetro en las soldaduras/conexiones.

R01F01T01P02 – Verificar encendido de LEDs de estado en RFID y ESP32 al alimentar el Arduino.

R01F01T02 – Programación del firmware en C++ para la lectura de tarjetas (Tarea 2).

R01F01T02P01 – Compilación del código sin errores de sintaxis en el IDE.

R01F01T02P02 – Visualización del UID de una tarjeta en el Monitor Serie al acercarla al lector.

R01F01T03 – Establecer conectividad P2P inicial entre el chip ESP32 y el ordenador de desarrollo (Tarea 3).

R01F01T03P01 – Ejecutar un comando ping desde el ordenador a la IP asignada al ESP32 y recibir respuesta (0% packet loss).

R01F01T04 – Implementar protocolo de transferencia de datos del Arduino al ordenador/servidor (Tarea 4).

R01F01T04P01 – Enviar una cadena de texto de prueba desde el Arduino y verificar su recepción correcta en un socket de escucha o terminal en el ordenador.

R01F01T05 – Sincronización de lista blanca: Transferir UIDs válidos de la BD al Arduino para validación local (Tarea 5).

R01F01T05P01 – Prueba de acceso positivo: Pasar una tarjeta registrada y verificar que el Arduino devuelve "Acceso Permitido".

R01F01T05P02 – Prueba de acceso negativo: Pasar una tarjeta no registrada y verificar que el Arduino devuelve "Acceso Denegado".

R02 – Infraestructura de Red Segura

Descripción: El sistema debe operar en una red segura, separando el tráfico público del privado y permitiendo el acceso remoto seguro para los administradores.

R02F01 – Segmentación de Red y Bastionado (VLAN/DMZ)

Descripción técnica: Despliegue de VLANs y configuración de firewall para aislamiento de la DMZ.

R02F01T01 – Creación de la VLAN y migración de la conectividad P2P a infraestructura de red gestionada (Tarea 6).

R02F01T01P01 – Verificar que el ESP32 obtiene una IP dentro del rango de la subred de la VLAN (ej. 192.168.20.x).

R02F01T01P02 – Intentar acceder al dispositivo desde una red no autorizada y verificar que el firewall bloquea la conexión.

R02F01T02 – Configuración del acceso VPN para entrada a la DMZ (Implícito en tu descripción de funciones).

R02F01T02P01 – Conectar desde una red externa (4G/casa) mediante el cliente VPN y realizar un ping al servidor interno.

R03 – Registro de Eventos y Auditoría Visual

Descripción: El sistema debe guardar la hora exacta de entrada y una foto del usuario en el momento del acceso para posterior verificación.

R03F01 – Backend y Almacenamiento de Datos

Descripción técnica: Servidor Ubuntu con pila LAMP/LEMP y base de datos relacional.

R03F01T01 – Instalación y bastionado del servidor Ubuntu, MySQL y Apache/Nginx (Docker) (Tarea 7).

R03F01T01P01 – Escaneo de puertos con Nmap para confirmar que solo los puertos necesarios (80, 443, 3306 interno) están abiertos.

R03F01T01P02 – Cargar la página "index.html" por defecto del servidor web desde un navegador cliente.

R03F01T02 – Diseño y creación del esquema de Base de Datos: Tablas para Usuarios, Logs y RFIDs (Tarea 8).

R03F01T02P01 – Ejecutar script SQL de inserción de datos (INSERT) y comprobar mediante un SELECT que los datos persisten correctamente.

R03F02 – Frontend y Sistema de Cámaras

Descripción técnica: Interfaz web PHP y scripts de captura de imagen RTSP/IP.

R03F02T01 – Desarrollo de la interfaz web con roles diferenciados (Admin/Empleado) (Tarea 9).

R03F02T01P01 – Intentar acceder al panel de administración con credenciales de empleado y verificar que el acceso es denegado o el menú está oculto.

R03F02T02 – Programación PHP para visualizar comparativa: Foto de base de datos vs. Foto del evento (Tarea 10).

R03F02T02P01 – Verificar que, al cargar el log, si la imagen del evento no existe, se muestra un "placeholder" o mensaje de error controlado, en lugar de romper la página.

R03F02T02P02 – Comprobar visualmente que la foto mostrada a la izquierda (BD) coincide con el nombre del usuario del registro.

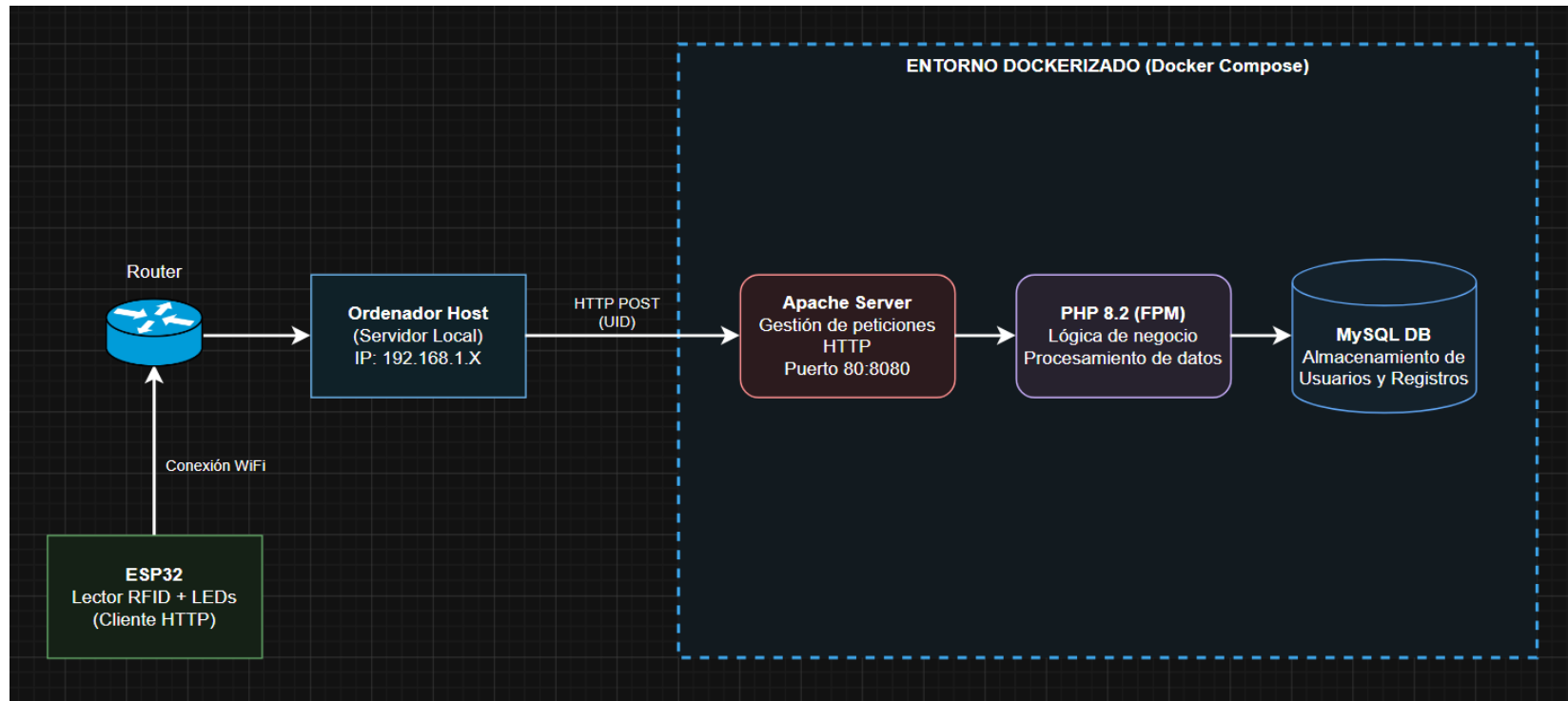
R03F02T03 – Conexión de cámara IP y script de captura de "snapshot" al activarse el RFID (Tarea 11).

R03F02T03P01 – Simular evento de entrada y verificar que en la carpeta del servidor se crea un archivo .jpg nuevo con la marca de tiempo actual.

R03F02T03P02 – Prueba de latencia: Mover la mano frente a la cámara al activar el sensor y verificar que la foto no sale movida o con un retraso excesivo.

DESCRIPCIÓN

Arquitectura de la solución.



Casos de uso. Incluye diagrama y tabla con:

Caso de uso: Iniciar sesión

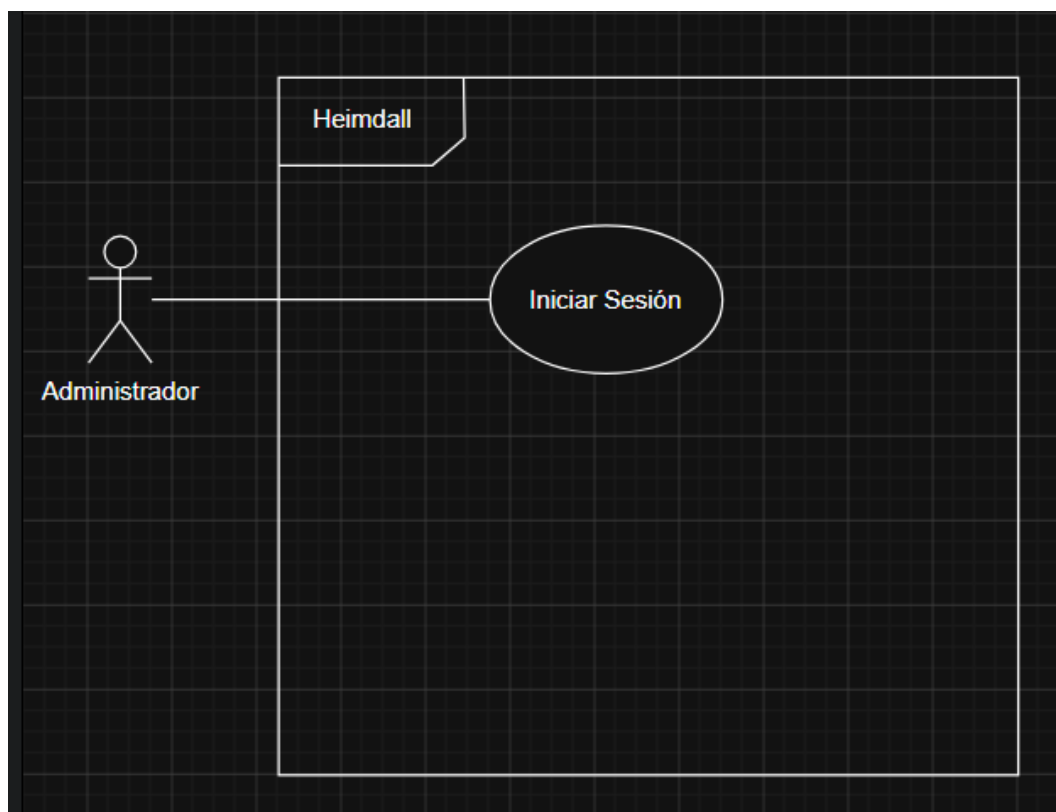


Ilustración 1: Caso de uso: Iniciar sesión

DESCRIPCIÓN: El administrador se autentica en la aplicación web para acceder al panel de control.	
PRECONDICIONES: El usuario debe existir en la base de datos de administradores.	POSTCONDICIONES: Se crea la sesión y se redirige al panel principal (index.php).
DATOS ENTRADA	DATOS SALIDA

Usuario, Contraseña.	Mensaje de error (si falla) o acceso concedido.
TABLAS: administradores	CLASES:
INTERFACES:	

Ilustración 2 Caso de uso: Inicio de Sesión

Caso de uso: Monitorear Accesos en Tiempo Real

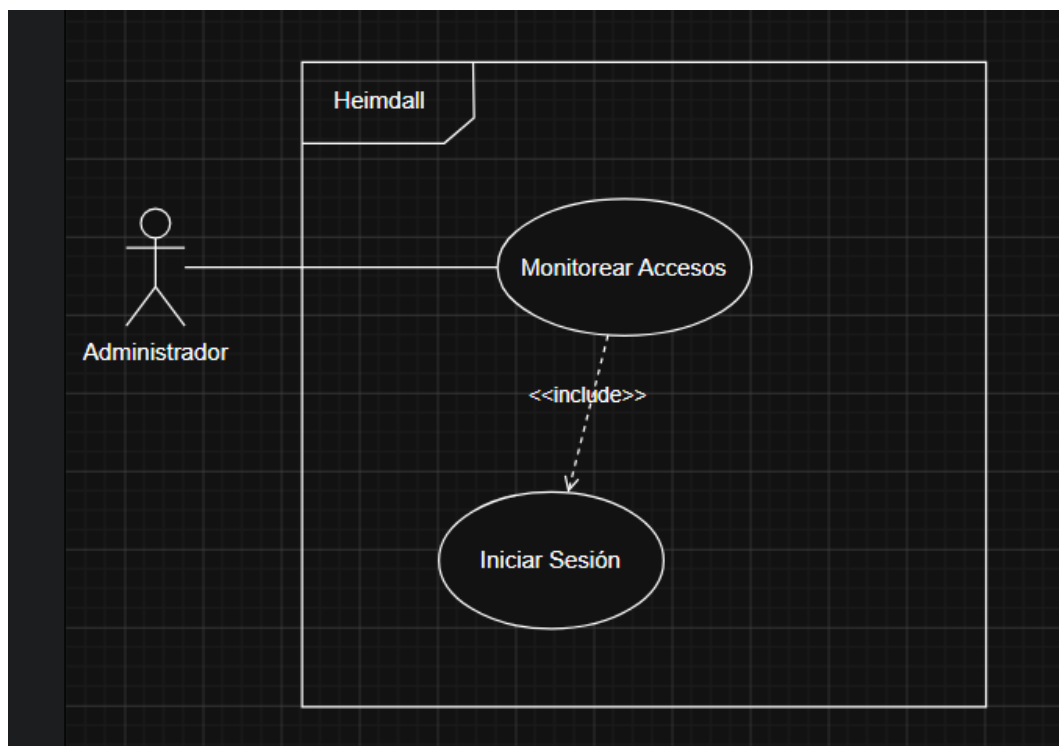


Ilustración 3 Caso de uso: Monitorear Accesos en Tiempo Real

DESCRIPCIÓN: El administrador visualiza en el panel (dashboard) la información de la persona que acaba de pasar la tarjeta.	
PRECONDICIONES: Administrador logado en el sistema.	POSTCONDICIONES: Visualización en pantalla de los datos actualizados del empleado.
DATOS ENTRADA Ninguno	DATOS SALIDA Foto, Nombre, Apellido, Cargo, UID de tarjeta, Hora y Fecha de entrada.
TABLAS: accesos, tarjetas, usuarios	CLASES: index.php, ajax_accesos.php
INTERFACES:	

Ilustración 4 Caso de uso: Monitorear Accesos en Tiempo Real

Caso de uso: Consultar Historial de Accesos

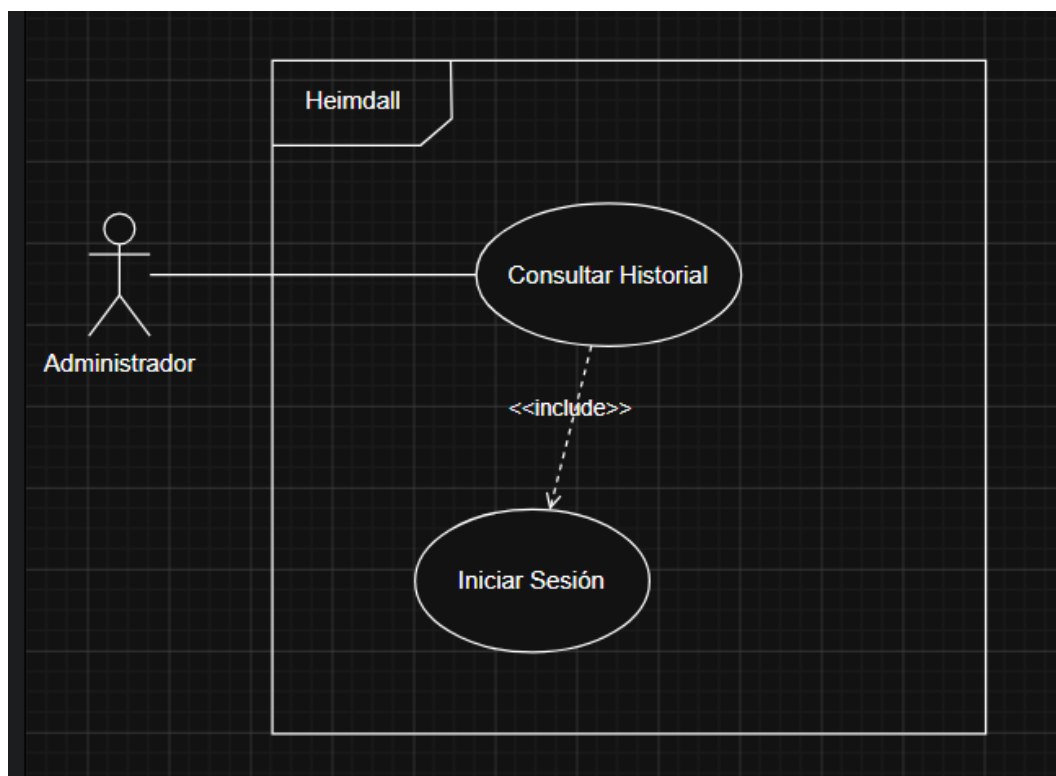


Ilustración 5 Caso de Uso: Consultar Historial de Accesos

DESCRIPCIÓN: El administrador revisa la lista de los accesos pasados de los empleados.

PRECONDICIONES:

Administrador logado en el sistema.

POSTCONDICIONES:

Se muestra una vista con el listado histórico de entradas y salidas.

DATOS ENTRADA

Clic en la sección de historial, posibles filtros (fechas, nombre).

DATOS SALIDA

Lista con: Nombre, UID, Fecha, Hora y Tipo de acceso.

TABLAS: accesos, tarjetas, usuarios.	CLASES: historial.php
INTERFACES:	

Ilustración 6 Caso de Uso: Consultar Historial de Accesos

Caso de Uso: Alta de Nuevo Usuario y Tarjeta

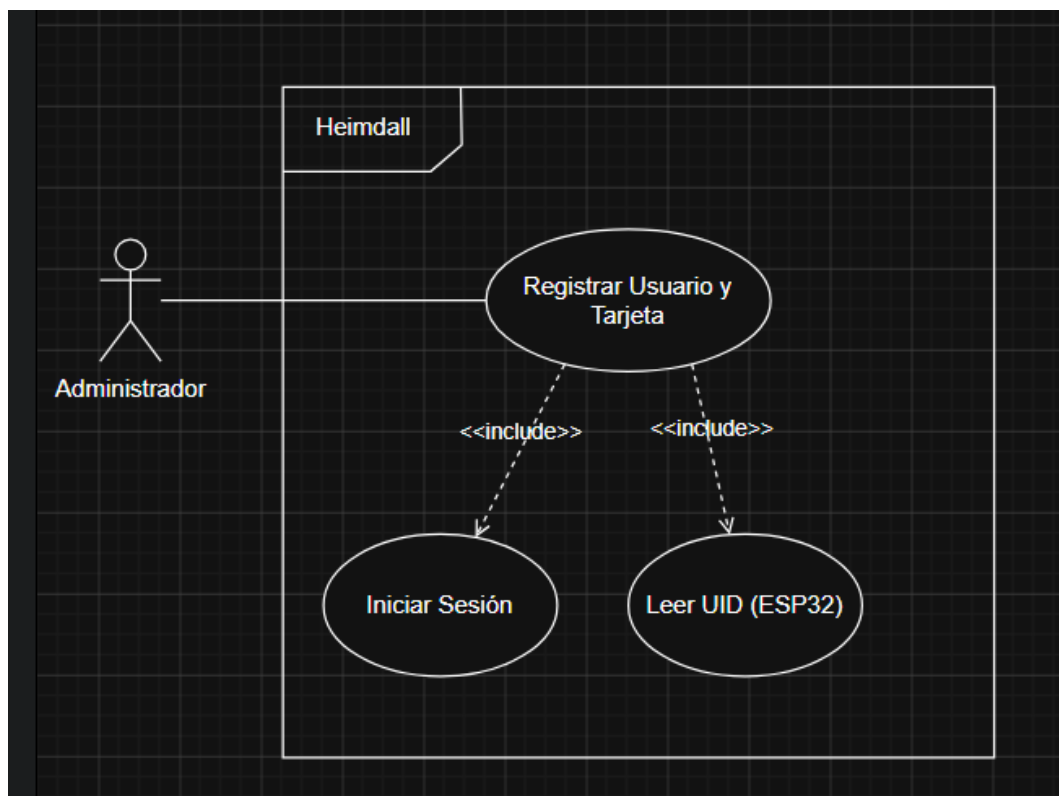


Ilustración 7 Caso de Uso: Alta de Nuevo Usuario y Tarjeta

DESCRIPCIÓN: El administrador registra a un nuevo empleado y le vincula un UID de tarjeta RFID para darle acceso.

PRECONDICIONES: Administrador logado en el sistema.	POSTCONDICIONES: Nuevo registro de usuario y tarjeta insertados en la base de datos.
DATOS ENTRADA Nombre, Apellido, Documento, Cargo, Foto, UID de la tarjeta.	DATOS SALIDA Mensaje de éxito o error al guardar.
TABLAS: usuarios, tarjetas.	CLASES: registro_tarjetas.php
INTERFACES:	

Ilustración 8 Caso de Uso: Alta de Nuevo Usuario y Tarjeta

Caso de Uso: Cerrar Sesión

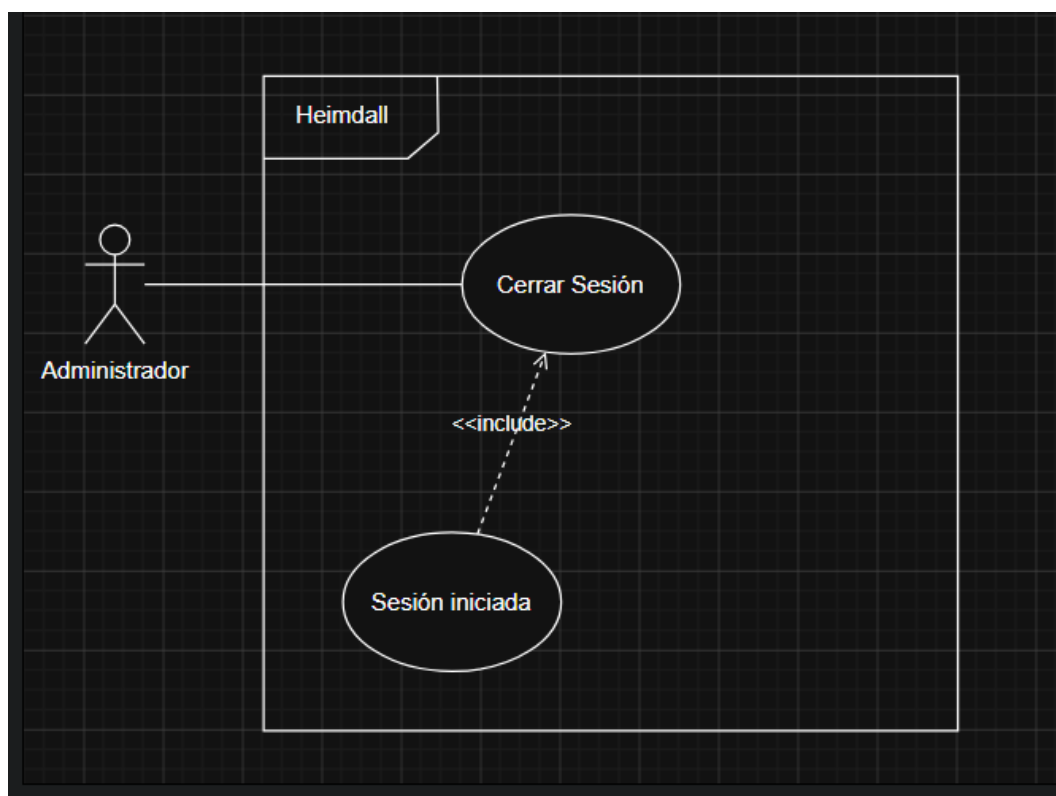


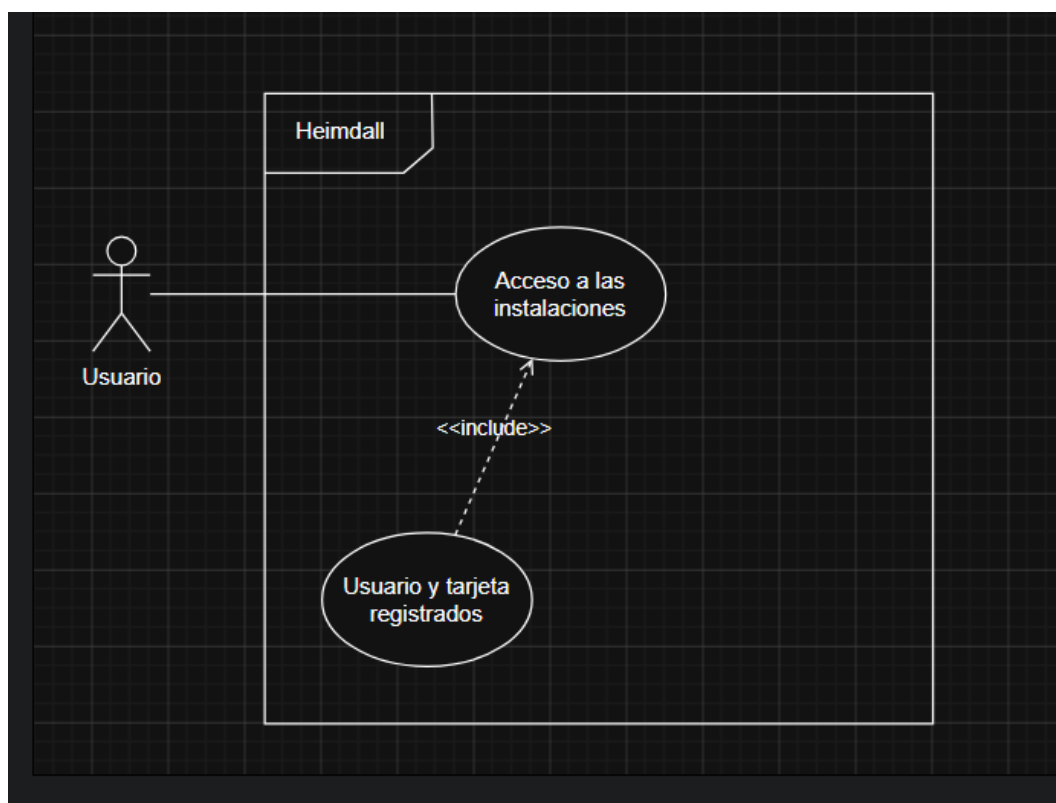
Ilustración 9 Caso de Uso: Cerrar Sesión

DESCRIPCIÓN: El administrador finaliza su sesión segura para evitar accesos no autorizados al panel.	
PRECONDICIONES: Administrador logado.	POSTCONDICIONES: Destrucción de la sesión y redirección a la pantalla de login.
DATOS ENTRADA Clic en el botón "Cerrar sesión".	DATOS SALIDA Pantalla de inicio de sesión.
TABLAS:	CLASES:

Ninguna	logout.php.
INTERFACES:	

Ilustración 10 Caso de Uso: Cerrar Sesión

Caso de Uso: Fichar



DESCRIPCIÓN: Un empleado acerca su tarjeta al lector RFID (Heimdall) para intentar acceder físicamente al recinto.

PRECONDICIONES:

Tener una tarjeta física.

POSTCONDICIONES:

Registro del intento en la base de datos

	y feedback visual (LED) del resultado.
DATOS ENTRADA Aproximación de la tarjeta física (captura del UID).	DATOS SALIDA Señal visual en el dispositivo (LED encendido si se permite, apagado si no).
TABLAS: tarjetas, usuarios, accesos.	CLASES: Lector ESP32 (.ino), validar.php.
INTERFACES:	

DISEÑOS

Diagrama E/R (Entidad - Relación)

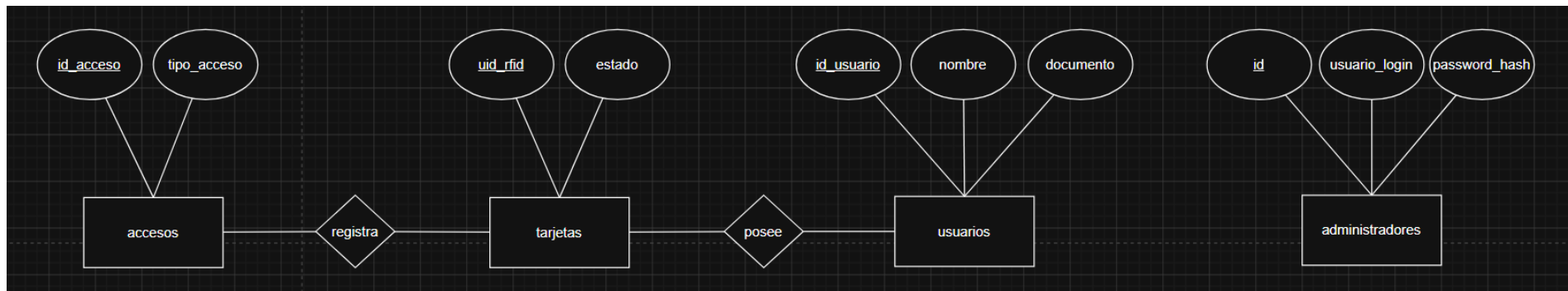


Ilustración 11 Diagrama E/R

Diagrama de la base de datos. Con detalle de campos.

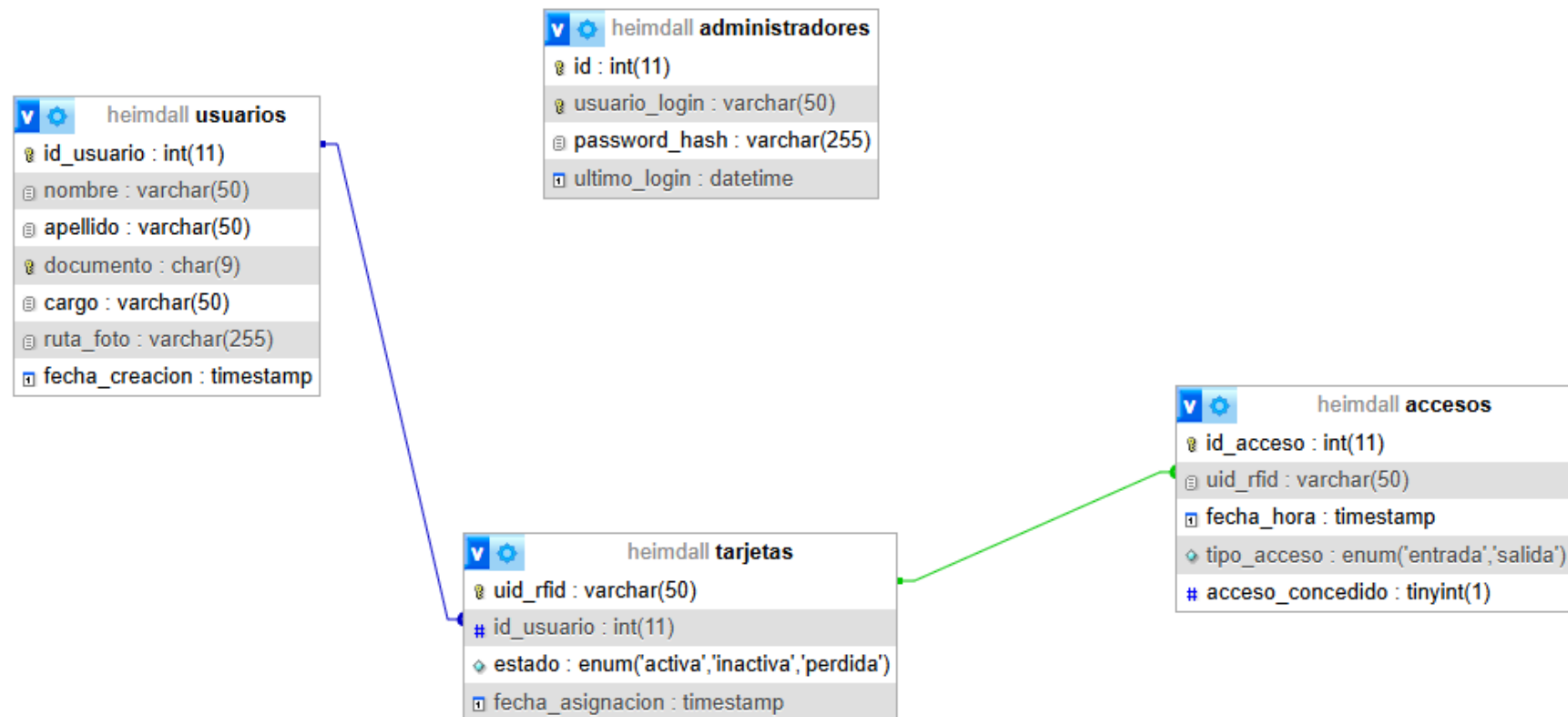


Ilustración 12 Tablas desde MySQL

Interfaces. Interesa ver la solución en diferentes tamaños o dispositivos.

PANEL DE CONTROL

ÚLTIMO ACCESO REGISTRADO



• ACCESO CONCEDIDO

Adrian Canton

ESTUDIANTE

Hora 01:07:50

Fecha 2026-05-05

Ilustración 13 Interfaz gráfica del proyecto

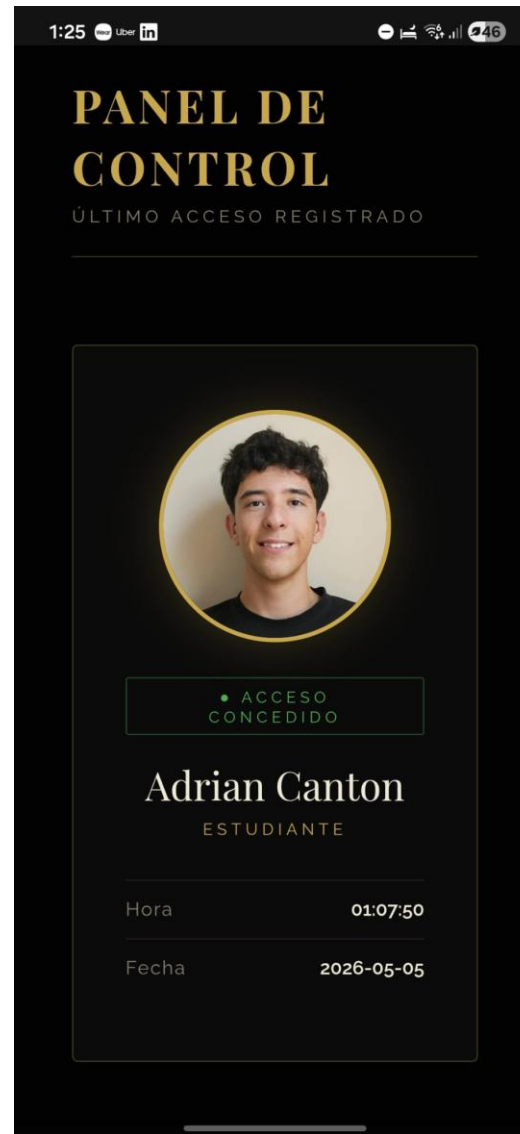


Ilustración 14 Interfaz grafica desde movil

TECNOLOGÍA

Las tecnologías y herramientas utilizadas para este proyecto. Por ejemplo:

LENGUAJES



PHP 8.3.

Orientado al desarrollo web, interpretado y de tipado dinámico, ampliamente usado en aplicaciones que requieren interacción con bases de datos..

En el proyecto: constituye el núcleo del backend. Gestiona la autenticación de administradores (login.php), valida las tarjetas RFID recibidas del ESP32 (validar.php), consulta la base de datos y sirve el panel de control en tiempo real (index.php, ajax_accesos.php).



C++

De bajo nivel con extensiones orientadas a objetos, utilizado en el ecosistema Arduino para programar microcontroladores mediante una API simplificada sobre C++ estándar.

En el proyecto: es el lenguaje del firmware del ESP32. Gestiona la inicialización del lector RFID, la lectura del UID de cada tarjeta, la conexión WiFi y el envío de la petición HTTP POST al servidor.

SQL

Estándar para la definición, manipulación y consulta de



bases de datos relacionales.

En el proyecto: se usa en dos ficheros: 01-setup.sql define el esquema completo de la base de datos (tablas usuarios, tarjetas, accesos, administradores, claves foráneas y restricciones), y 02-data-test.sql inserta datos de prueba para el desarrollo.

HTML / CSS / JavaScript

Tecnologías estándar del frontend web. HTML define la estructura, CSS el estilo visual y JavaScript el comportamiento dinámico en el navegador.



En el proyecto: el HTML y CSS embebidos en los archivos PHP dan forma al panel de control y al login. El JavaScript implementa el polling AJAX cada 1,5 segundos que actualiza el dashboard sin recargar la página.

BASE DE DATOS



MySQL 8.4.

Sistema de gestión de bases de datos relacional de código abierto, uno de los más utilizados en aplicaciones web. Soporta transacciones, claves foráneas e integridad referencial.

En el proyecto: almacena las cuatro entidades principales del sistema: usuarios registrados, tarjetas RFID con su estado (activa / inactiva / perdida), registros de acceso con marca de tiempo y tipo (entrada / salida), y credenciales de

administradores. La base de datos se llama Heimdall.

HARDWARE



ESP32.

Microcontrolador de 32 bits con WiFi y Bluetooth integrados, ampliamente utilizado en proyectos de IoT por su bajo coste, bajo consumo y capacidad de conectarse a redes IP sin hardware adicional.

En el proyecto: actúa como pasarela entre el lector RFID y el servidor web. Se conecta a la red WiFi local, lee el UID de la tarjeta aproximada al lector y lo envía al endpoint `validar.php` mediante una petición HTTP POST.

MFRC522

Módulo lector/escritor de tarjetas RFID que opera en la frecuencia de 13,56 MHz, compatible con tarjetas del estándar ISO 14443A (las tarjetas Mifare más comunes). Se comunica con el microcontrolador mediante el protocolo SPI.

En el proyecto: es el sensor físico que detecta y lee las tarjetas RFID. Extrae el UID único de cada tarjeta y lo pasa al ESP32 para su posterior envío al servidor.



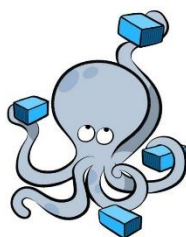
INFRAESTRUCTURA Y DESPLIEGUE



Docker.

Plataforma de contenerización que permite empaquetar aplicaciones y sus dependencias en contenedores aislados, garantizando que el entorno de ejecución sea idéntico en cualquier máquina.

En el proyecto: orquesta los tres servicios del sistema mediante `docker-compose.yml`: el servidor web Apache, el procesador PHP-FPM y la base de datos MySQL, cada uno en su propio contenedor.



Docker Compose

Herramienta de Docker para definir y ejecutar aplicaciones multi-contenedor mediante un fichero YAML de configuración declarativa.

En el proyecto: define la arquitectura completa del sistema en un único fichero: imágenes a construir, variables de entorno, volúmenes, dependencias entre servicios y segmentación de redes. Permite levantar todo el stack con un único comando (y `docker compose up -d`).



Apache HTTP Server 2.4

Servidor web de código abierto, uno de los más utilizados del mundo.

En este proyecto actúa como servidor de archivos estáticos y proxy inverso hacia PHP-FPM.

En el proyecto: recibe las peticiones HTTP del navegador y del ESP32, sirve los archivos estáticos (imágenes de usuarios) y redirige las peticiones .php al contenedor PHP mediante el protocolo FastCGI (fcgi://php:9000).



PHP-FPM (FastCGI Process Manager)

Implementación alternativa del gestor de procesos PHP, diseñada para entornos de alta carga. Separa el servidor web del intérprete PHP en procesos independientes que se comunican mediante FastCGI.

En el proyecto: ejecuta los scripts PHP en un contenedor separado del de Apache, mejorando el aislamiento y la seguridad. Apache le delega las peticiones PHP a través de la red interna Docker.



Ubuntu Server

Distribución Linux de servidor basada en Debian, ampliamente usada como sistema operativo anfitrión para entornos de producción y desarrollo por su estabilidad y soporte a largo plazo.

En el proyecto: es el sistema operativo de la máquina anfitriona donde corre Docker. El script `instalar_docker.sh` automatiza la instalación de Docker sobre Ubuntu.

Documentación

Infraestructura

Para que el proyecto sea totalmente portable hemos usado **Docker y Docker compose**. De manera que levanta un contenedor por cada servicio (uno para Apache, otro PHP y MySQL). Además, en Docker creamos 2 redes separadas:

1. Red interna: Esta sirve para conectar los servicios entre sí, (en esta red esta Apache, PHP y MySQL)
2. Red externa: Aquí solo se expone el servicio de Apache en el puerto deseado para poder acceder a el mediante web

Dockerfile

Hemos creado nuestros propios Dockerfile donde parcheamos las vulnerabilidades de las imágenes más fiables, además ejecutamos todos los usuarios como **no root** para que en caso de ciberataque no puedan entra a otros servicios a parte del del afectado.

Docker compose

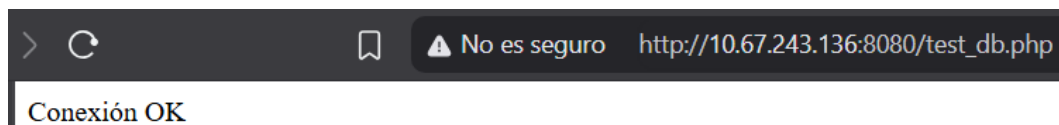
En este archivo se usa para levantar los Dockerfile mencionados anteriormente pero de forma **personalizada** es decir, creando nuestra propia base de datos e incluyendo nuestro código en PHP. Además de crear los volúmenes persistentes necesarios.

Conexiones

MySQL

Se levanta en un contenedor en la red interna que hemos creado y cuando abre el puerto 3306 al levantarse el contenedor, permite la conexión de PHP.

Y gracias a un código simple que envía una petición a la base de datos, desde PHP podemos ver si es correcta la conexión haciendo una simple pregunta de si el “estado” es “conexión ok”:



PHP

Para que funcione la conexión de MySQL con PHP necesitamos un archivo que haga la conexión. Este recoge la información al crearse el contenedor usando las mismas variables que para la base de datos

Para que Apache funcione perfectamente y nos muestre la pagina de PHP tiene que conectarse a través de un archivo que llamábamos **vhost.conf**:

Le indicamos el archivo que queremos mostrar como index y donde se encuentra todo el PHP

ESP32

La “conexión” de este microcontrolador es a través de la librería WiFi.h, esta funciona añadiéndole el nombre de la red y la contraseña. Además de añadirle las páginas a las que tiene que enviar peticiones. Todo ello securiado con un token que comprueba que el que envia peticiones **sí es el ESP32**

Página web

Esta se divide en index, registrar, historial y tarjetas

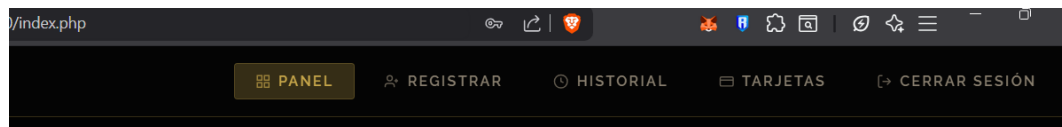


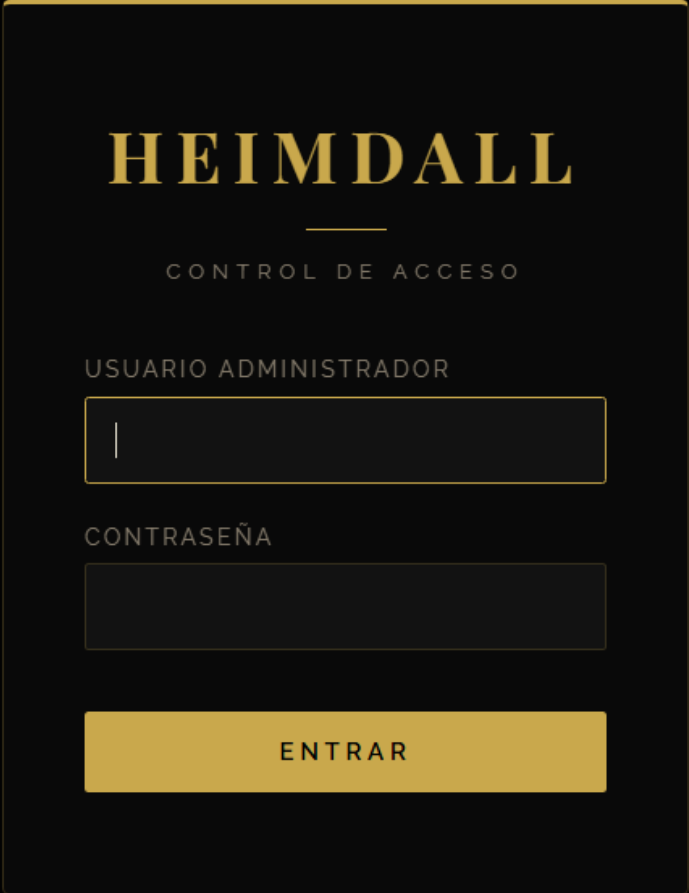
Ilustración 15 barra de navegación

Login

Antes de poder entrar en cualquier página tienes que pasar por un login, en este el usuario ya tiene que estar creado de antes, aquí se ve como sin el login no puedes acceder al index.

El login.php es un formulario básico que comprueba si existe el usuario y contraseña en la tabla administradores donde las contraseñas se guardan encriptadas.

Y el login se veía así:



The image shows a login interface for a system named 'HEIMDALL'. The title 'HEIMDALL' is displayed in a large, bold, gold-colored serif font. Below it, the subtitle 'CONTROL DE ACCESO' is shown in a smaller, gold-colored sans-serif font. The form contains two input fields: 'USUARIO ADMINISTRADOR' and 'CONTRASEÑA', both with gold borders. A gold 'ENTRAR' button is positioned at the bottom of the form. The entire interface is set against a dark background.

Ilustración 16 login

Índex

En esta página se muestra un nav-bar (barra de navegación) para poder navegar por todo el sitio web y además está constantemente en escucha para comprobar nuevos accesos:

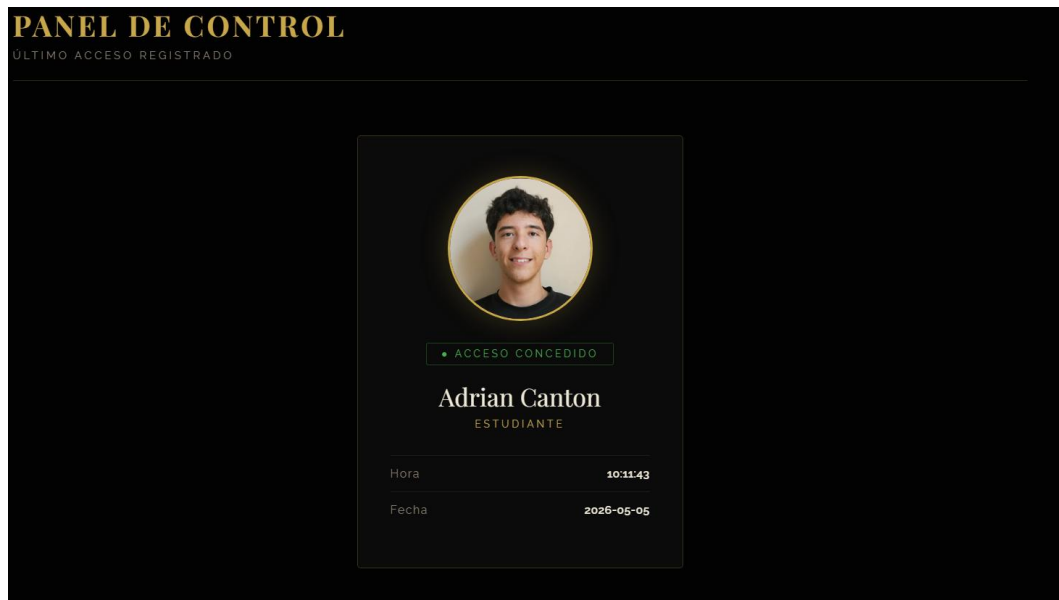


Ilustración 17 index

Registrar

Se trata de un formulario pero con un campo especial, ya que tenemos un modo en el ESP32 donde lee el ID de las tarjetas y lo manda al campo de ID tarjeta. Realmente como funciona es haciendo un POST cambiando un txt por el último ID leído:

The image shows a web form titled "ALTA DE PERSONAL" with the subtitle "REGISTRO DE NUEVO USUARIO Y TARJETA RFID". The form is set against a dark background. It contains several input fields: "NOMBRE" and "APELLIDO" (both empty), "DNI / DOCUMENTO" (empty), and "CARGO" (empty). Below these is a section for "FOTOGRAFÍA DEL EMPLEADO" with a button labeled "SELECCIONAR ARCHIVO" and the text "Ningún archivo seleccionado". Further down is a section for "UID TARJETA RFID" showing the value "F778C3A2" and a confirmation message "✓ Tarjeta detectada — F778C3A2". At the bottom of the form is a large yellow button labeled "GUARDAR REGISTRO".

Ilustración 18 registro

Historial y tarjetas

Estas páginas son una simple petición a la base de datos de las tablas de tarjetas y accesos donde genera una tarjeta para cada dato

Y quedan tal que así

REGISTRO DE TARJETAS

USUARIOS REGISTRADOS Y SUS TARJETAS RFID ASIGNADAS

11
TOTAL USUARIOS

8
TARJETAS ACTIVAS

3
SIN TARJETA / INACTIVAS

Buscar por nombre, DNI o UID...

USUARIO	DNI	UID TARJETA	ESTADO	ASIGNADA	REGISTRO
 Adrian Canton ESTUDIANTE	123456790	F778C3A2	ACTIVA	05/05/2026	05/05/2026
 Tom Nook JEFE	999999990	578E2625	ACTIVA	05/05/2026	05/05/2026
 Elon Musk DIRECTOR	12345678	4A:5B:6C:7D	ACTIVA	05/05/2026	05/05/2026

Ilustración 19 logs

HISTORIAL DE ACCESOS

REGISTRO COMPLETO DE ENTRADAS Y SALIDAS

8
TOTAL REGISTROS

7
CONCEDIDOS

1
DENEGADOS

TIPO: Todos ESTADO: Todos FECHA: dd/mm/aaaa

FILTRAR LIMPIAR

#	USUARIO	UID TARJETA	TIPO	ESTADO	FECHA	HORA
8	 Adrian Canton Estudiante	F778C3A2	↑ ENTRADA	✓ CONCEDIDO	05/05/2026	10:11:43
7	 Tom Nook Jefe	578E2625	↑ ENTRADA	✓ CONCEDIDO	05/05/2026	10:11:39

Ilustración 20 historial de accesos

Páginas no visibles

Conexión.php

Este archivo es el **más importante de todos** ya que es el que se encarga de que haya una conexión entre la base de datos y el servidor PHP.

Funciona de forma que accede a través de la red de Docker (por eso no hay IP de host) intentando el usuario y contraseña, dados anteriormente, por el puerto 3306

[Logout.php](#)

Esta página se encarga de borrar el token de sesión creado al iniciar sesión en login.php

[Validar.php](#)

Se encarga de comprobar que el ID de la tarjeta que envía el ESP32 exista en la tabla “tarjetas” de la base de datos

[Ajax accesos.php](#)

Se encarga de estar constantemente comprobando si hay un nuevo acceso, mostrando el ultimo dato de la tabla de “accesos”

[Microcontroladores](#)

La forma de enviar todo lo que pasamos por la tarjeta RFID es a través de API REST que no son más que consultas MySQL enviadas a través de un POST.

Todas ellas securizadas con el **token** en la cabecera, si no el PHP no responde.

Hemos creado dos códigos dependiendo de si necesitas añadir un usuario o si necesitas validar un usuario.

Hardware

Para ello hemos usado un ESP32 junto a un RFID para crear el hardware de control de acceso, a este se le puede meter el código que necesites (Registrar/Validar). Como veis en la siguiente imagen:

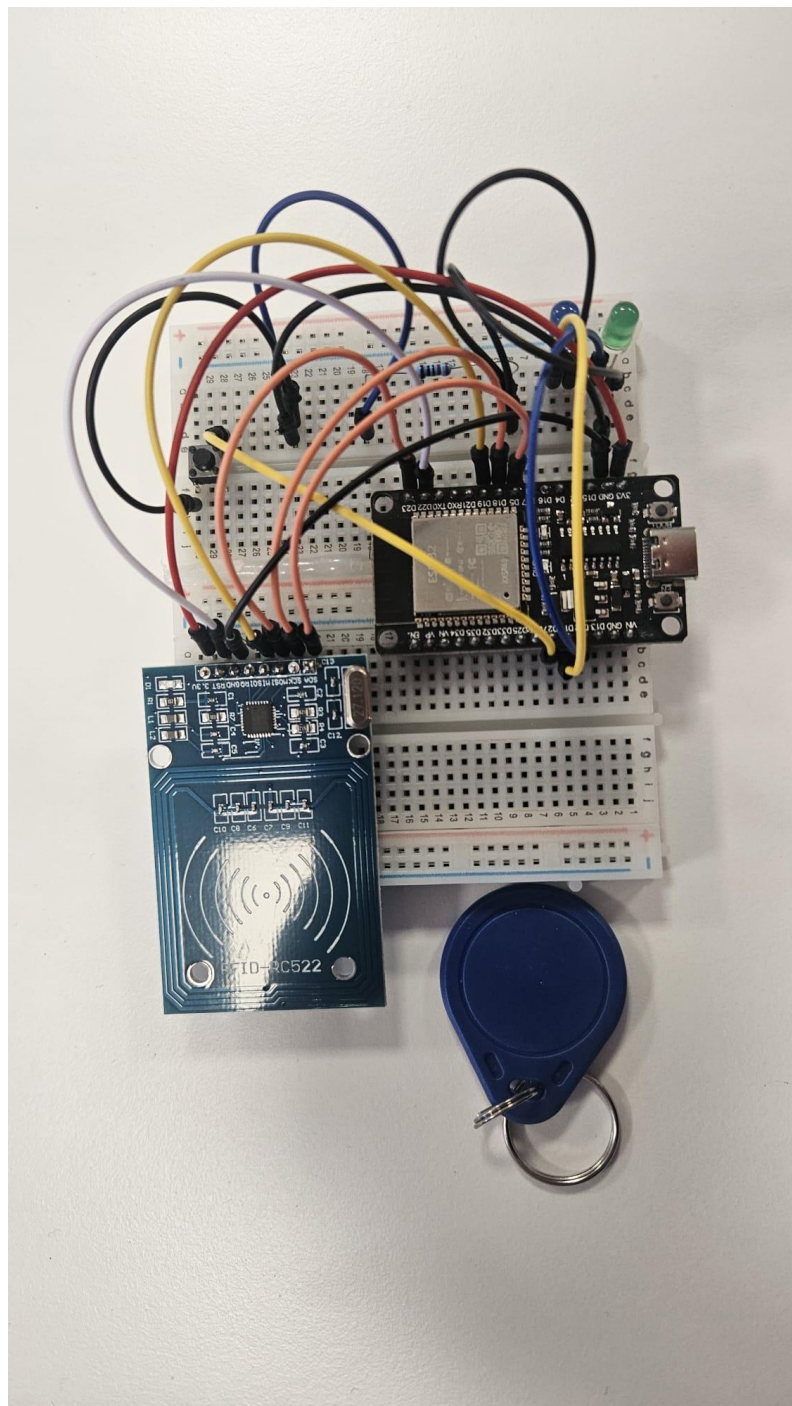


Ilustración 21 Circuito de Heimdall

Registro

En este caso enviamos el ID de la tarjeta leída a un formulario para crear una nueva inserción en usuarios.

Validación

Aquí, en lo que nos centramos con la API REST es en comprobar si existe el ID de la tarjeta en la base de datos.

METODOLOGÍA

Valoración inicial

TAREAS
Ensamblaje de arduino
Programación RFID
Conectividad arduino-PC
Transferencia info. Arduino-PC
Transferencia info. PC-Arduino
Montar Ubuntu
Crear las tablas de BD
Hacer la pag web
Hacer PHP de la pag web

Ilustración 22 tareas propuestas

Semana	Día	Fecha	Horas	Visual
Semana 1	J	26/02/2026	2H	[Color Azul]
Semana 1	V	27/02/2026	3H	
Semana 1	S	28/02/2026	0H	[Color Negro]
Semana 1	D	01/03/2026	1H	
Semana 2	L	02/03/2026	2H	[Color Verde]
Semana 2	M	03/03/2026	2H	
Semana 2	X	04/03/2026	1H	
Semana 2	J	05/03/2026	2H	[Color Verde Claro]
Semana 2	V	06/03/2026	1H	
Semana 2	S	07/03/2026	0H	[Color Negro]
Semana 2	D	08/03/2026	0H	

Ilustración 23 aproximacion de horas s1 y s2

Semana	Día	Fecha	Horas	Visual
Semana 3	L	09/03/2026	2H	
Semana 3	M	10/03/2026	3H	
Semana 3	X	11/03/2026	2H	
Semana 3	J	12/03/2026	2H	
Semana 3	V	13/03/2026	3H	
Semana 3	S	14/03/2026	0H	
Semana 3	D	15/03/2026	1H	
Semana 4	L	16/03/2026	3H	
Semana 4	M	17/03/2026	2H	
Semana 4	X	18/03/2026	3H	
Semana 4	J	19/03/2026	2H	
Semana 4	V	20/03/2026	2H	
Semana 4	S	21/03/2026	0H	
Semana 4	D	22/03/2026	1H	

Ilustración 24 aproximación de horas s1 y s2

Tiempo real

- Ensamblaje de Arduino: 3H
- Programación RFID: 25H
- Conectividad arduino-PC: 1H
- Transferencia info. Arduino-PC: 2H
- Transferencia info. PC-Arduino: 1H
- Montar Ubuntu (Docker al final): 20H
- Crear las tablas de BD: 2H
- Hacer la pag web: 1H

- Hacer PHP de la pag web: 30H
- Documentación: 10H

Total: 95 horas

Presupuesto

Aproximadamente el sector de innovación cobra unos 50 -200 € la hora dependiendo de como de grande es la idea y quien es el contribuidor, teniendo esto en cuneta veo justificable un precio de 55€/h con un total de:

$$95 \times 55 \text{ €} = 5.225 \text{ €}$$

A ello le tenemos que sumar un aproximado de 30% de beneficio (pero en nuestro caso vamos a aproximarlos a un 20%):

- Gestión del proyecto
- Pruebas
- Riesgos
- Soporte mínimo
- Beneficio

Lo que daría un presupuesto final de $5.225 \text{ €} \times 1,20 = 6.270 \text{ €}$

Que junto a los materiales utilizados: microcontrolador ESP32, Lector RFID, Tarjetas/pllaveros, Cableado le sumaríamos unos +180€

Entre **6.300 € – 6.500 € + IVA**

TRABAJOS FUTUROS

Trabajos de ampliación y mejora proyectados.

Autenticación y autorización mejoradas

JWT (JSON Web Tokens) en lugar de tokens estáticos: con firma criptográfica y expiración automática

OAuth 2.0 / OpenID Connect para el panel web (Login moderno).

Cifrado extremo a extremo

HTTPS con TLS también entre ESP32 y backend (no solo web). Con certificados cliente (mTLS) para los ESP32: Además cada dispositivo tiene su propio certificado. Mucho más seguro que tokens.

Protección ante ataques

Rate limiting por IP y por dispositivo.

Detección de intentos de acceso anómalos (RFID brute-force).

Evolución tecnológica e innovación

Biometría con una integración futura de huella

MFA físico como RFID + biometría

CONCLUSIONES

El desarrollo del proyecto Heimdall ha permitido consolidar una solución de control de acceso que equilibra de manera eficiente la seguridad física con la agilidad tecnológica. A través de la integración de hardware de bajo coste y una arquitectura de software moderna, se ha demostrado que es posible mitigar las vulnerabilidades de los sistemas tradicionales de llaves sin incurrir en los costes prohibitivos de las soluciones propietarias.

Más allá de la funcionalidad técnica, el proyecto resuelve un problema logístico crítico: la trazabilidad. La capacidad de monitorizar en tiempo real y auditar mediante capturas visuales transforma un simple punto de paso en un nodo de información inteligente, proporcionando a los administradores una herramienta de control total sobre el flujo de personas.

REFERENCIAS

Infraestructura y Contenerización

Aplicado para la arquitectura de servicios, el aislamiento del sistema y la orquestación del despliegue mediante contenedores.

Docker Documentation. (2025). Docker Compose overview. Recuperado de <https://docs.docker.com/compose/>

Microcontroladores y Firmware

Aplicado en el desarrollo del firmware, la gestión de la conectividad WiFi y el control de los periféricos de la placa.

Espressif Systems. (s. f.). ESP32 Series Documentation. Recuperado de <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>

Tecnología de Identificación (RFID)

Aplicado para la integración del lector MFRC522, la gestión del protocolo SPI y la lectura de los identificadores de las tarjetas.

Balboa, M. (2023). MFRC522 Library for Arduino and ESP32. Recuperado de <https://github.com/miguelbalboa/rfid>

Desarrollo Web y Lógica de Servidor

Aplicado para la creación de la lógica de negocio, el dashboard administrativo y la gestión de peticiones asíncronas.

The PHP Group. (2024). PHP Manual: Language Reference. Recuperado de <https://www.php.net/docs.php>

Servidor de Aplicaciones

Aplicado para la configuración y administración del servidor web que aloja el sistema y gestiona el tráfico de datos.

The Apache Software Foundation. (2024). Apache HTTP Server Documentation. Recuperado de <https://httpd.apache.org/docs/2.4/>

Gestión de Bases de Datos

Aplicado para la persistencia de datos, la gestión de usuarios y la seguridad de las consultas mediante sentencias preparadas.

Oracle Corporation. (2025). MySQL 8.0 Reference Manual. Recuperado de <https://dev.mysql.com/doc/refman/8.0/en/>